

Data preprocessing document

Tech-Triathlon 2026 Datathon · Team AlphaZero · Tasks 1, 2A and 2B

Every step below runs in `AlphaZero_FinalNotebook.ipynb` (section numbers in brackets). All checks are written as assertions, so the notebook stops if the data ever break an assumption. **No rows are deleted in any task.**

1. Data preparation at a glance

Task	Files used	Unit of one training row	Rows
1 · service time and lateness	<code>deliveries_train</code> + <code>route_legs_train</code> (actual times); test: <code>task1_test_inputs</code> + <code>route_legs_test</code> (planned times only); <code>outlets</code> , <code>vehicles</code> , <code>service_allowance</code> , <code>calendar</code> , <code>road_conditions</code> , <code>traffic_speed</code>	one dispatched order = one stop	91,894 train, 5,014 test
2A · weekly demand	<code>deliveries_train</code> + <code>task1_test_inputs</code> (order columns only), <code>calendar</code> , <code>outlets</code>	depot × brand × ISO week	702 history (6 series × 117 weeks), 60 to forecast
2B · peak-day allocation	<code>task2b_peak_day_scenarios</code> , <code>task2b_peak_day_fleet</code> , <code>vehicles</code> , <code>district_travel</code> , <code>service_allowance</code>	one order of scenario S1	85 orders, 38 vehicles

Integrity checks (all asserted, §2.3–§2.6, §9, §12.3). `delivery_id` unique across files; one order per outlet × date × temperature; no missing demand fields; volume, weight and units positive; every order date is in the calendar and is an operating day; brand, district and depot agree with `outlets.csv`; chilled orders only for Fresh; `dispatch_date` blank only for never-dispatched (`not_run`) orders and never before `order_date`; the calendar is daily, gap-free, ISO-correct, with Sundays closed; every dispatched order and every test order matches exactly one route leg.

2. Task 1 · service time and lateness (§2–§7)

Merge. Each dispatched order is its own stop: it joins its route leg one-to-one on `route_id` + `seq_in_route` (pandas `validate="one_to_one"`), and the leg's outlet, date, vehicle, brand, district and depot must agree with the order (asserted). This holds even when a route stops twice at the same outlet (621 legs). The 413 `not_run` orders have no leg, so no label. Clock times are converted to minutes after midnight; a malformed value fails a check.

Label construction. Both targets come from the actual times in the training route legs, which never exist at prediction time:

Label	Definition	Trap avoided
<code>service_min</code>	<code>leave_outlet_time - max(arrival_time, window_open_time)</code>	4.4% of vehicles arrive before the window opens and wait about 20 min; raw dwell time would count that wait as handling
<code>is_late</code>	<code>arrival_time > window_close_time</code> (strictly)	arrivals exactly at closing time are on time (late rate 19.58%, not 19.88%)

Cleaning. 87 integrity and label checks pass (§2.3–§2.6), including a deterministic planned timeline (each planned departure = previous planned arrival + allowance). Service times are skewed (median 15 min, maximum 428); the long stops are real Style and Tech deliveries, so they are kept.

Feature engineering (91 service / 96 lateness), all known before departure.

- **Plan and context (47 + 8 categorical):** order size and density, dock, access, mall, allowance, window, stop position and stops left, planned slack and wait, cumulative planned travel and distance, route load, traffic at the leg's planned hour, the day's road disruption and the travel time they imply, calendar.

- **As-of history `h_*` (31):** outlet, outlet × monsoon, recent 12 weeks, vehicle, district × stop × monsoon, district travel ratio, depot departure delay, and minutes per unit / kg, computed only from labelled rows **strictly before a cutoff**: the start of the row's six-week block in training, the fold start in validation, 16 Feb 2026 for the test. History is therefore 0–6 weeks old everywhere, as it is for the test. Outlet means are shrunk toward their brand × dock prior.
- **ETA replay `eta_*` (10):** every planned route is replayed stop by stop with expected travel (traffic × disruption × the district's historical ratio), waits for windows to open and expected (size-aware) service, giving an expected arrival, delay and slack for each stop.

Actual-time columns appear only inside the history aggregates; `feature_list()` refuses them as features, and rebuilding the features after deleting every row from the cutoff on gives identical values.

Rationale. The exploratory analysis (§3) shows outlet history carries forward in time, order size drives handling, and lateness is accumulated travel delay along the route; each family maps to one finding, and each earns its place in the ablations (§6.3). Validation mirrors the test: six-week time folds, each trained only on earlier data, plus two folds that no decision used (§4, §6.11).

3. Task 2A · weekly depot demand (§8–§22)

Label construction. The target of a row is the volume of all orders of one depot and brand whose `order_date` falls in one ISO week (`iso_year`, `iso_week` from `calendar.csv`); the chilled target is the same sum over `temp_requirement == "chilled"`. §10.1 measures what each rule protects against:

Rule	If ignored
bucket by <code>order_date</code> , not <code>dispatch_date</code>	95 of 702 weekly rows change, 1,337 m³ in the wrong week
keep <code>deferred</code> and <code>not_run</code> orders (they are demand)	138 rows change, 3,701 m³ lost, mostly chilled (5.4% of chilled orders)
append <code>task1_test_inputs</code> as history	a six-week gap right before the forecast
take ISO weeks from the calendar; chilled = 0 for Style and Tech	periods or chilled values that do not match the brief

Cleaning. All 97,321 orders kept. The only known gap: the six weeks from `task1_test_inputs` contain no `not_run` orders; the order-slot reconstruction below restores them. Series panels are complete grids (a week without orders is 0, not a missing row), with daily and weekly totals checked to agree.

Feature engineering. Two model families use two feature views:

- **Order slots × order size** (§12.2). Every outlet orders on fixed weekdays (Fresh ambient daily, Fresh chilled on four weekdays, Style on one, Tech on two eligible days about half the time). The learned schedules rebuild all 117 historical weeks exactly for Fresh ambient and Style; an order on a closed day is lost, not moved. The size of each order uses weekday, a 3-day payday window, the festival ramp **and which festival it leads to** (New Year ≈ +80%, Thai Pongal ≈ +19%), trend, annual seasonality per brand × temperature, and the outlet's own level.
- **Series panels at a forecast origin** (§12.3). Levels from “clean” weeks (6 operating days, no festival), weekday factors, payday uplift, festival slopes and year-on-year growth, all computed only from weeks up to the origin. A leakage test replaces every later outcome with random numbers and requires identical features.

Excluded on evidence: monsoon and road disruption (§11.6: no effect on order volume; they affect travel, which is Task 1).

Rationale. Weekly volume = a known number of order slots × a calendar-driven order size, so the model predicts order size and sums the slots of each forecast week. The choice between the two model families is made per brand on a 49-origin rolling backtest (§18).

4. Task 2B · peak-day allocation (§23)

Preparation. Each order is joined to its handling allowance (brand × dock type) and its district's outbound and inter-stop free-flow minutes. Trip minutes = outbound + (n – 1) × inter-stop + Σ allowances, exactly as the brief and `check_allocation.py` compute them. The 10 vehicles in the workshop are removed.

Cleaning and pre-filtering. Refrigeration, van-only access and home-depot rules remove impossible order–vehicle pairs before optimisation. One order can never be served (§1-078: 40.7 m³ against a 38 m³ largest truck, and orders cannot be split); it is flagged with its reason rather than dropped.

Rationale. Refrigerated capacity is the binding resource (181.6 m³ of chilled demand against 86.2 m³ per reefer wave), so reefers carry only chilled orders and the priorities are applied by an exact optimiser (written policy: `docs/Task2B_Prioritization_Policy.md`).